A Non-Compact Formulation and Complexity Framework for the Tool Switching Problem

Shankar Narayanan

Internship Report

Advisors: Prof. Rafael Colares and Prof. Annegret Wagler

March 30, 2026

Abstract

The management of tool configurations in Flexible Manufacturing Systems (FMS) is a well-studied NP-hard combinatorial optimization challenge. This report provides a mathematical analysis of the sequential Tool Switching Problem (SSP) by evaluating its non-compact formulation. We systematically establish the structural equivalences of the SSP to classical optimization frameworks, specifically mapping its continuous routing to the Generalized Traveling Salesman Problem (GTSP) and its configuration bounds to the Job Grouping Problem (JGP). Furthermore, we establish a strict distinction between the Uniform SSP (U-SSP) and the Tool-Dependent SSP (TD-SSP), proving that the JGP serves as the exact operational objective for the latter via Lexicographical Minimization. Finally, we demonstrate that the dynamic generation of these configurations within an exact Branch-and-Price framework is mathematically equivalent to the Set Union Knapsack Problem (SUKP), explicitly necessitating Machine Learning interventions to accelerate the NP-hard pricing oracle.

1 Introduction

Flexible Manufacturing Systems (FMS) enable rapid adaptation to market changes by utilizing numerically controlled machines equipped with automated tool magazines. Because a single machine may be required to process a diverse batch of parts, and because the total number of distinct tools required often exceeds the capacity of the machine's magazine, tool switches are unavoidable (Tang and Denardo, 1988). A tool switch incurs a time penalty

that delays production. Consequently, optimal tool management is a primary objective in manufacturing optimization.

This report focuses on the Job Sequencing and Tool Switching Problem (SSP). In this problem, a set of jobs must be processed on a single machine equipped with a tool magazine of limited capacity. Each job requires a specific subset of tools, and all required tools must be present in the magazine before the job can be processed. The objective is to determine a processing sequence for the jobs and an associated tool loading policy that jointly minimize the total cost or number of tool switches.

Because the SSP is strongly NP-hard (Crama and Oerlemans, 1994), modern solution methodologies rely on decomposition. As formally proven by Tang and Denardo (1988), the SSP naturally decomposes into two nested subproblems: the **Sequencing Problem (SP)** (determining the optimal job order) and the **Tooling Problem (TP)** (determining the optimal tool loading and eviction policy for a fixed sequence).

To accurately model the computational complexity of the TP, the literature strictly distinguishes between two variants of the problem, which we will formally define and utilize throughout this report:

- **Uniform SSP (U-SSP):** The standard variant where the time or effort required to change any tool is identical (a uniform cost of 1). The objective is to minimize the absolute *number* of individual tool switches.
- **Tool-Dependent SSP (TD-SSP):** A complex real-world variant where setup times are non-uniform and vary significantly depending on the specific tool being inserted or removed.

To develop a state-of-the-art exact solver, it is imperative to move beyond isolated heuristics and situate the SSP within the broader landscape of Combinatorial Optimization (CO). The primary objective of this report is to analyze the non-compact formulation of the SSP by proving its mathematical equivalence to other classical CO domains, thereby justifying an advanced algorithmic architecture.

The remainder of this report is structured as follows. Section 2 introduces the non-compact mathematical formulation, scoped strictly to the U-SSP. Section 3 maps the continuous routing state space of the U-SSP to the Generalized Traveling Salesman Problem (GTSP). Section 4 analyzes the configuration bounds provided by the Job Grouping Problem (JGP), and resolves the operational differences between the U-SSP and TD-SSP via Lexicographical Minimization. Section 5 formalizes the exact Branch-and-Price pricing oracle as the Set Union Knapsack Problem (SUKP). Finally, Sections 6 and 7 outline cutting-edge Machine Learning pathways—such as Neural Column Generation—designed to bypass this NP-hard pricing bottleneck.

2 The Non-Compact Formulation for the U-SSP

In order to solve the Uniform SSP exactly, we must first map its state space. Traditionally, compact Integer Linear Programming (ILP) formulations attempt to model the exact placement of individual tools in the magazine slots at each discrete time step. However, to overcome the severe symmetry issues inherent in those compact models, state-of-the-art approaches utilize a non-compact formulation based on routing between maximal tool configurations.

Scope of Analysis: Throughout Section 2, Section 3, and Sections 4.1–4.3, we operate strictly under the **Uniform SSP (U-SSP)** assumption. This means that the time or effort required to insert or remove any individual tool is identical. This assumption is mathematically critical because it guarantees that the cost of transitioning between any two states of the machine is symmetric and exactly proportional to the discrete number of exchanged tools.

The computational complexity of the SSP is fundamentally rooted in its state space. Traditional compact models sequence the jobs directly and track tool insertions step-by-step. However, these models suffer from symmetry caused by 0-blocks. As formally defined by Crama and Oerlemans (1994), a 0-block is a maximal subset of consecutive zeros in a row of the tool-job incidence matrix—representing a time interval before and after which a tool is needed, but during which it is not required.

The act of retaining an unneeded tool in the magazine across this gap to avoid redundant unloading and reloading is mathematically referred to as "flipping" the 0-block to 1s. This structure forces compact solvers to evaluate highly symmetric, redundant permutations to decide which 0-blocks to flip, severely limiting computational performance (Catanzaro et al., 2015).

2.1 Illustrative Example: Magazine State Evolution

To illustrate the tool switching mechanics and the concrete impact of flipping a 0-block, consider the 5-job, 7-tool instance. The machine has a magazine capacity of $b = 4$, and the jobs are processed in the sequence $j_1 \to j_2 \to j_3 \to j_4 \to j_5$. The specific tool requirements for each job are illustrated in the incidence matrix in Figure 1.

The specific tool requirements for each job are as follows:

- $T_1 = \{t_1, t_2, t_3\}$
- $T_2 = \{t_2, t_3, t_4\}$
- $T_3 = \{t_4, t_5, t_6\}$

	j_1	j_2	j_3	j_4	j_5
t_1	1	0	0	1	1
t_2	1	1	0	0	1
t_3	1	1	0	0	0
t_4	0	1	1	0	0
t_5	0	0	1	1	0
t_6	0	0	1	1	0
t_7	0	0	0	0	1

Figure 1: An incidence matrix for a 5-job, 7-tool instance. The machine magazine has a capacity $b = 4$. Assuming the sequence $j_1 \to j_2 \to j_3 \to j_4 \to j_5$, tool t_1 is required by jobs j_1, j_4, and j_5, but not by j_2 and j_3. This gap of non-requirement represents a 0-block for t_1. Because $b = 4$ is large enough to accommodate the transition tool requirements, the optimal policy evaluates whether to "flip" this 0-block by retaining t_1 through jobs j_2 and j_3 to avoid redundant setups.

- $T_4 = \{t_1, t_5, t_6\}$
- $T_5 = \{t_1, t_2, t_7\}$

Under the optimal Keep Tool Needed Soonest (KTNS) policy (Tang and Denardo, 1988), the solver looks ahead to determine which 0-blocks to flip to minimize future swaps. The evolution of the optimal magazine configurations across this sequence is depicted in Figure 2.

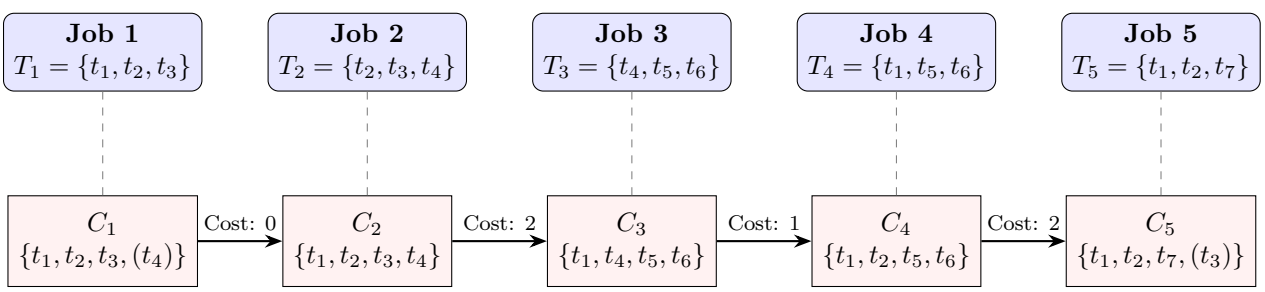

Figure 2: Evolution of the optimal magazine configurations across the job sequence with capacity $b = 4$. Notice how tool t_1 is retained in configurations C_2 and C_3 despite not being required, successfully flipping the 0-block to save future tool switches.

To bypass the sequence-based symmetry inherent in determining which 0-blocks to flip step-by-step, we utilize the non-compact formulation proposed by Colares and Wagler (2026), which projects the problem entirely onto a complete directed graph of valid configurations.

2.2 Definitions, Variables, and Maximal Configurations

Let $J = \{1, 2, \ldots, n\}$ be the set of jobs and $T = \{1, 2, \ldots, m\}$ be the set of all available tools. Each job $j \in J$ requires a specific subset of tools $T_j \subseteq T$. The machine's tool magazine has a capacity b.

A fundamental property of this formulation is the assumption of maximal configurations. Let $\mathcal{C} = \{C \subseteq T \mid |C| \leq b\}$ be the collection of all valid tool configurations. Without loss of generality, we establish that the tool repository is always full, i.e., $|C| = b$ for all considered configurations (Colares and Wagler, 2026).

Proposition 2.1 (Configuration Maximization)**.** *Any configuration C where $|C| < b$ is strictly dominated by a maximal configuration $C' = C \cup T_{dummy}$ where $|C'| = b$.*

Proof. Let $d(C_1, C_2)$ be the number of tool switches required to transition from configuration C_1 to C_2. Under the U-SSP assumption, the cost formula evaluates to $d(C_1, C_2) = b - |C_1 \cap C_2|$. If $|C_1| < b$, the empty slots can be filled with arbitrary tools from $T \setminus C_1$ to form C_1', yielding $|C_1'| = b$. Because $C_1 \subseteq C_1'$, it mathematically follows that $|C_1 \cap C_2| \leq |C_1' \cap C_2|$. Therefore, $b - |C_1' \cap C_2| \leq b - |C_1 \cap C_2|$. Padding an under-capacity configuration incurs a mathematical penalty of zero while strictly increasing the probability that the added tools may satisfy subsequent job requirements. □

Let $H_j \subseteq \mathcal{C}$ denote the subset (cluster) of configurations that are capable of processing job $j \in J$. That is, $H_j = \{C \in \mathcal{C} \mid T_j \subseteq C\}$. The U-SSP can now be seen as finding the shortest sequence of configurations in a complete directed graph K_p, where $V(K_p) = \mathcal{C}$, visiting at least one vertex from each cluster H_j.

Let $y(v)$ denote a binary variable stating whether or not configuration $v \in V(K_p)$ is used to carry some job. Let $x(u, v)$ denote a binary variable stating whether or not configuration $v \in V(K_p)$ is applied right after configuration $u \in V(K_p)$. The U-SSP can thus be modeled as follows:

$$
\begin{aligned}
\min \quad & \sum_{u \in V} \sum_{v \in V} d(u,v) x(u,v) && (1)\\
\text{subject to} \quad & \sum_{v \in H_j} y(v) \geq 1, \quad \forall j \in J && (2)\\
& \sum_{u \in V} x(u,v) = y(v), \quad \forall v \in V && (3)\\
& \sum_{u \in V} x(v,u) = y(v), \quad \forall v \in V && (4)\\
& \sum_{u \in H(S)} \sum_{v \in H(S)} x(u,v) \leq \sum_{v \in H(S)} y(v) - 1, \quad \forall S \subseteq J, 2 \leq |S| \leq |J| - 1 && (5)\\
& x(u,v) \in \{0,1\}, \quad \forall u,v \in V && (6)\\
& y(v) \in \{0,1\}, \quad \forall v \in V && (7)
\end{aligned}
$$

where $H(S) = \bigcup_{j \in S} H_j$.

To establish the correctness of this formulation, we must prove that the constraints mathematically enforce a valid, continuous sequence of configurations that covers all jobs, and that the objective evaluates the optimal tool switching cost without loss of generality.

Lemma 2.1 (Flow and Cover Feasibility)**.** *Constraints (3), (4), and (5) ensure that the selected configurations form a valid, continuous routing sequence without disconnected sub-cycles.*

Proof. Constraints (3) and (4) enforce strict flow conservation for any selected configuration ($y(v) = 1$). Constraint (5) operates as the Generalized Subtour Elimination Constraint (GSEC). For any proper subset of jobs S, the number of internal transitions within the configurations covering S cannot exceed the total number of configurations covering S minus 1, strictly preventing closed sub-cycles disjoint from the main tour. □

Lemma 2.2 (Subtour Elimination)**.** *Constraint (5) guarantees that the active configurations form exactly one connected Hamiltonian cycle.*

Proof. Suppose, for the sake of contradiction, that a feasible solution satisfying (2)–(5) forms $k \geq 2$ disjoint directed cycles, $W_1, W_2, \ldots, W_k$. Let $V(W_1)$ be the set of active vertices in cycle W_1. Let $S \subset J$ be the proper subset of jobs covered exclusively by the configurations in W_1. Because $k \geq 2$ and the clusters H_j partition the job requirements, W_1 cannot process all jobs without rendering the other cycles completely redundant. Thus, $2 \leq |S| \leq |J| - 1$.

By definition, all vertices in W_1 belong to $H(S) = \bigcup_{j \in S} H_j$, meaning $V(W_1) \subseteq H(S)$. If we sum the selected edges $x(u, v)$ within W_1, we have:

$$\sum_{u \in V(W_1)} \sum_{v \in V(W_1)} x(u, v) = |V(W_1)| = \sum_{v \in V(W_1)} y(v) \tag{8}$$

Since $V(W_1) \subseteq H(S)$, evaluating the Subtour Elimination Constraint (5) strictly over S yields:

$$\sum_{u \in H(S)} \sum_{v \in H(S)} x(u, v) \geq \sum_{v \in V(W_1)} y(v) \tag{9}$$

However, constraint (5) strictly bounds the sum of the edges from above, demanding $\sum_{u \in H(S)} \sum_{v \in H(S)} x(u, v) \leq \sum_{v \in H(S)} y(v) - 1$. This constitutes a direct contradiction. Thus, the formulation strictly forbids disjoint sub-cycles, enforcing $k = 1$. □

Theorem 2.3 (Correctness)**.** *The integer linear program defined by (1)–(7) correctly identifies an optimal job sequence and tool-loading policy that minimizes the total number of tool switches for the U-SSP.*

Proof. By Lemma 2.1 and Lemma 2.2, the formulation's constraints define a single, connected cycle over a subset of configuration vertices that collectively satisfy all job requirements. For any two consecutively applied configurations C_u and C_v (where $x(u, v) = 1$), the number of tools that must be inserted into the magazine is exactly the capacity b minus the number of tools retained from the previous configuration, $|C_u \cap C_v|$. Thus, the transition edge cost $d(u, v) = b - |C_u \cap C_v|$ exactly computes the number of physical tool switches required. The objective function (1) accumulates these transition costs along the active edges of the cycle. Therefore, minimizing this objective over the feasible space correctly yields a sequence mapping directly to the minimum number of tool switches. □

3 Graph Transformations and Complexity

Having defined the configuration graph for the U-SSP, this section explores its theoretical equivalence to the Generalized Traveling Salesman Problem (GTSP). Mapping the continuous routing state space into an established routing framework is the critical first step toward building an exact solver. To formally analyze the algorithmic pathways for solving the U-SSP, we must establish its computational relationship with the GTSP. While the problems share a covering-and-routing structure, their state spaces differ fundamentally. This section details the graph transformations that link the two problems, formalizes the exponential complexity of these mappings, and establishes the complexity hierarchy via structural subsumption.

3.1 Formal Definition of Instances

We formally define an instance of the sequential U-SSP as $\mathcal{I}_{\text{U-SSP}} = \langle J, T, b, \{T_j\}_{j \in J} \rangle$, where $J = \{1, \ldots, n\}$ is the set of jobs, $T = \{1, \ldots, m\}$ is the set of tools, b is the magazine capacity, and $T_j \subseteq T$ is the specific tool requirement for each job.

Conversely, we define an instance of the GTSP as $\mathcal{I}_{GTSP} = \langle V, E, d, \{V_k\}_{k \in K} \rangle$, where V is the vertex set partitioned into mutually exclusive clusters V_k ($V_i \cap V_j = \emptyset$), E is the set of directed edges, and $d : E \to \mathbb{R}^+$ is the transition cost function. The objective is to find a minimum-cost cycle visiting exactly one vertex from each cluster.

3.2 State-Space Transformation: From U-SSP to GTSP

In the U-SSP, a single maximal tool configuration $C \in \mathcal{C}$ can contain the necessary tools to satisfy multiple jobs simultaneously. Therefore, the configuration clusters H_j are inherently overlapping ($H_i \cap H_j \neq \emptyset$). To map this overlapping state space into a disjoint routing problem, we adapt the Intersecting-to-Nonintersecting (I-N) transformation (Lien et al., 1993).

The Construction: We construct the transformed vertex set V' by duplicating valid U-SSP configurations into job-specific replicas. These vertices are partitioned into exactly $n = |J|$ clusters. Each cluster V_j corresponds uniquely to a job $j \in J$:

$$V_j = \{v_{C,j} \mid C \in \mathcal{C},\ T_j \subseteq C\} \quad \forall j \in J \tag{10}$$

Because each replica vertex is uniquely indexed by its job j, the transformed clusters are strictly disjoint ($V_i \cap V_j = \emptyset$ for $i \neq j$). The distance function d' between any two replicas in V' is defined by the underlying transition cost of their configurations:

$$d'(v_{C_u,i}, v_{C_w,j}) = b - |C_u \cap C_w| \tag{11}$$

Crucially, intra-cluster edges connecting replicas of the exact same configuration are assigned a cost of zero. This transformation is illustrated in Figure 3.

Lemma 3.1 (Non-Redundant Subtours)**.** *If a GTSP tour visits two replicas of the same configuration non-consecutively, it can always be transformed into a non-redundant tour of equal or lesser cost where all replicas of that configuration are visited consecutively.*

Proof. Let $v_{C,i}$ and $v_{C,j}$ be replicas of the same configuration C. Their internal distance is $d'(v_{C,i}, v_{C,j}) = b - |C \cap C| = 0$. Because the distance metric intrinsically satisfies the triangle inequality (Lemma 2.1), bypassing intermediate nodes to visit $v_{C,j}$ immediately after $v_{C,i}$ via

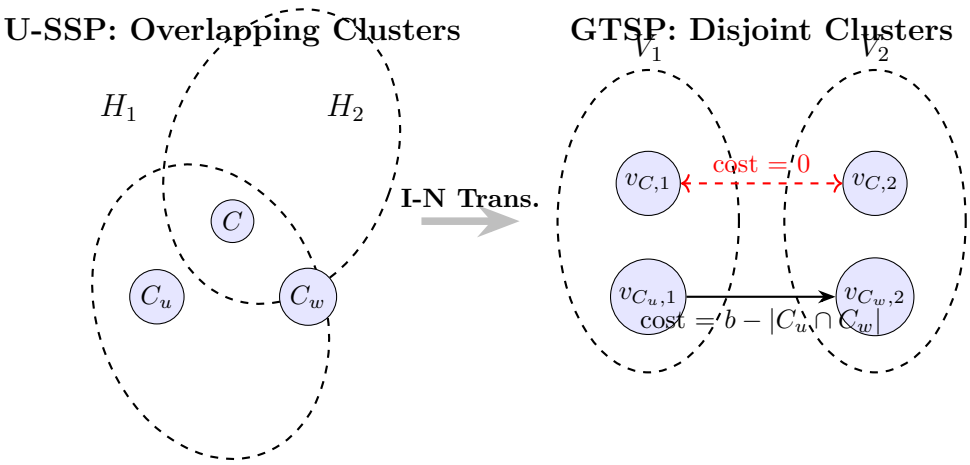

Figure 3: The Intersecting-to-Nonintersecting (I-N) transformation. A configuration C capable of satisfying both Job 1 and Job 2 exists in the intersection of H_1 and H_2. In the transformed graph, C is duplicated into job-specific replicas $v_{C,1}$ and $v_{C,2}$ in strictly disjoint clusters, connected by a zero-cost edge.

the zero-cost edge will never increase the total tour cost. Repeated shortcutting groups all replicas of C into a continuous zero-cost subtour (Lien et al., 1993). □

Theorem 3.2 (Optimal Solution Correspondence)**.** *A minimum-cost GTSP tour on the transformed graph corresponds to an optimal U-SSP sequence without loss of optimality.*

Proof. By Lemma 3.1, an optimal GTSP tour can be made non-redundant. In a non-redundant tour, all replicas of a chosen configuration C are visited consecutively, incurring an internal transition cost of 0 between them. This perfectly emulates the Keep Tool Needed Soonest (KTNS) policy, which retains tool configurations across a 0-block of jobs at zero penalty (Tang and Denardo, 1988). Consequently, this non-redundant sequence of disjoint clusters collapses directly into a valid, sequence-preserving tool-loading policy for the U-SSP with identical cost. □

Complexity of the Transformation: While the I-N transformation establishes a theoretical correspondence, it is critical to quantify its algorithmic complexity. The maximum number of valid configurations is $\binom{m}{b}$. Replicating these across n jobs yields a transformed state space strictly bounded by $\mathcal{O}(n\binom{m}{b})$. Because this state-space expansion requires exponential time and space, it operates as an algorithmic formulation technique for routing solvers rather than a polynomial-time Karp reduction.

3.3 Theoretical Equivalence and Structural Subsumption

To investigate whether the U-SSP and GTSP are strictly equivalent complexity classes, one must analyze the reverse mapping: converting a GTSP into the U-SSP.

The Restricted Reverse Construction: A reverse mapping is theoretically possible under highly restricted conditions. If a GTSP instance possesses strictly integer, symmetric, and Euclidean edge weights, it can be mapped to the U-SSP by artificially engineering dummy tools.

Lemma 3.3 (Dummy Tool Padding)**.** *For any symmetric GTSP edge weight $w_{uv} \in \mathbb{Z}^+$ bounded by b, there exists a theoretical U-SSP configuration space where the transition cost matches the edge weight: $b - |C_u \cap C_v| = w_{uv}$.*

Proof. If we assign a unique, disjoint set of w_{uv} dummy tools to configurations C_u and C_v, while filling their remaining $b - w_{uv}$ slots with identical shared tools, the resulting set intersection is exactly $|C_u \cap C_v| = b - w_{uv}$. Applying the transition formula yields $b - (b - w_{uv}) = w_{uv}$, perfectly matching the edge weight. □

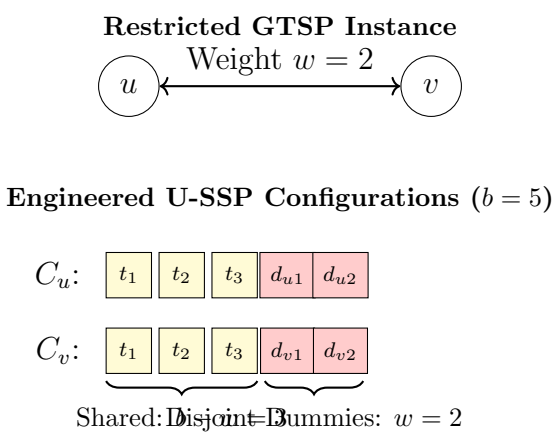

Figure 4: The theoretical restricted reverse construction. By generating disjoint dummy tools for specific configurations, the U-SSP transition metric $b - |C_u \cap C_v|$ can be artificially engineered to match integer GTSP edge weights. This construction requires exponential time to encode a full graph.

The Metric Barrier: While Lemma 3.3 demonstrates a mapping, a general polynomial reduction from the GTSP to the **U-SSP** is mathematically impossible. The barrier lies in the fundamental nature of the transition costs. As established, the U-SSP's transition metric ($b - |C_u \cap C_v|$) is strictly integer-bounded, inherently symmetric, and naturally satisfies the triangle inequality. In stark contrast, a general GTSP allows for arbitrary, real-valued, and highly asymmetric edge weights. It is mathematically impossible to polynomially construct

dummy tools whose set intersections can capture arbitrary real-valued metrics. However, it is worth noting that the Tool-Dependent SSP (TD-SSP) variant, which is often modeled as a max-flow min-cost network (Privault and Finke, 1995), *can* absorb asymmetric transition costs, foreshadowing its higher complexity.

Because a general reverse reduction is prohibited by the metric barrier, the formal complexity hierarchy between the two problems is established not by artificial graph transformations, but by structural subsumption. The GTSP can be seen as a particular case of the Tool Switching Problem where the clusters H_j are all disjoint (Colares and Wagler, 2026). We formally prove that this disjointness condition occurs naturally without any artificial edge-weight mapping.

Proposition 3.1 (The Disjointness Condition)**.** *If the union of the required tools for any two distinct jobs $i, j \in J$ strictly exceeds the magazine capacity, i.e., $|T_i \cup T_j| > b$, then their corresponding configuration clusters H_i and H_j are strictly disjoint ($H_i \cap H_j = \emptyset$).*

Proof. Suppose, for the sake of contradiction, that $|T_i \cup T_j| > b$ holds, yet the clusters are not disjoint ($H_i \cap H_j \neq \emptyset$). This implies there exists at least one valid tool configuration $C \in \mathcal{C}$ such that $C \in H_i$ and $C \in H_j$. By definition of the clusters, this requires $T_i \subseteq C$ and $T_j \subseteq C$. It mathematically follows that their union must also be a subset of the configuration: $(T_i \cup T_j) \subseteq C$. Consequently, the cardinality must satisfy $|T_i \cup T_j| \leq |C|$. Since every valid configuration is strictly bounded by the magazine capacity ($|C| = b$), we arrive at $|T_i \cup T_j| \leq b$. This directly contradicts the initial premise that $|T_i \cup T_j| > b$. □

Having established the structural subsumption and the continuous routing equivalence of the U-SSP to the GTSP, we have fully defined the state space required for an exact solver. We now turn to the problem of generating the discrete maximal configurations that populate this state space, exploring the tight mathematical bounds provided by the Job Grouping Problem (JGP).

3.4 Algorithmic Implications of the Complexity Space

Because the SSP is structurally more generalized and its forward state-space transformation (the I-N mapping) requires exponential memory, generating the complete configuration graph is only computationally viable for smaller, tightly-constrained instances. This inherent complexity dictates a bifurcated solution methodology (illustrated in Figure 5).

For tractable instances, the complete GTSP graph and its mutually exclusive clusters can be generated in memory. We then apply the Noon-Bean transformation (Noon and Bean, 1993), a graph transformation that modifies the edge weights to compel any optimal routing

algorithm to visit exactly one node per cluster. This collapses the generalized problem into a standard Asymmetric Traveling Salesman Problem (ATSP), which can be solved optimally using state-of-the-art routing heuristics such as LKH-3 (Helsgaun, 2015).

However, for massive industrial instances where the combination of m and b triggers out-of-memory failures, exactness requires bypassing the full graph via dynamic node generation in a Branch-and-Price paradigm.

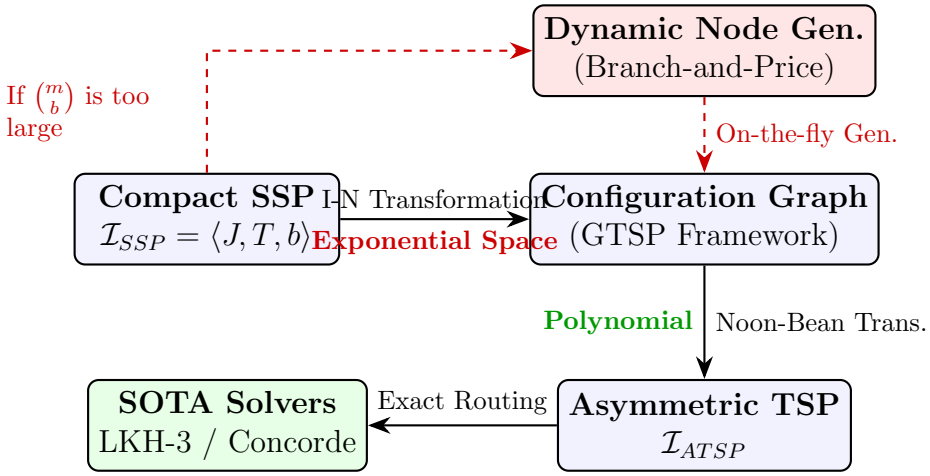

Figure 5: The complexity space of solving the SSP. Tractable instances can be reduced to ATSP and solved via highly optimized routing solvers. Massive instances bypass the exponential bottleneck via dynamic node generation in a Branch-and-Price framework.

4 Relationship with the Job Grouping Problem (JGP)

4.1 Conceptual Distinction and Suboptimality of JGP

The JGP aims to partition a set of jobs into the minimum number of feasible groups such that all jobs within a group can be processed with a single setup of the tool magazine. It assumes the machine is completely halted between groups to allow for simultaneous, batch tool swaps. In contrast, the Uniform SSP (U-SSP) seeks to determine a continuous processing sequence that minimizes individual tool switches, dynamically retaining unneeded tools across transitions via policies such as the Keep Tool Needed Soonest (KTNS) rule Tang and Denardo (1988).

Because the JGP forces jobs into rigid batches, it locks the magazine configuration for the duration of the group. This rigidity can temporarily choke the magazine capacity and force the unnecessary eviction of tools that will be required shortly after, leading to strict suboptimality when evaluated under continuous transitions. It is crucial to distinguish this

from the Tool-Dependent SSP (TD-SSP), where asymmetric setup costs often make full batching mathematically optimal Crama and Oerlemans (1994). However, for the U-SSP, rigid batching is strictly suboptimal.

To prove this, consider an instance with a magazine capacity of $b = 3$, containing 6 jobs and 6 tools that form a cyclical requirement ring:

- $T_1 = \{t_1, t_2\}$
- $T_2 = \{t_2, t_3\}$
- $T_3 = \{t_3, t_4\}$
- $T_4 = \{t_4, t_5\}$
- $T_5 = \{t_5, t_6\}$
- $T_6 = \{t_6, t_1\}$

The strict objective of the JGP is to partition the jobs into a minimum number of feasible groups. Because every 3 consecutive jobs require 4 distinct tools (e.g., $T_1 \cup T_2 \cup T_3 = \{t_1, t_2, t_3, t_4\}$), no 3 jobs can fit into a single group. The absolute minimum number of groups is 3 batches of 2 jobs each:

1. **Batch 1 (Jobs 1 & 2):** Requires configuration $\{t_1, t_2, t_3\}$.
2. **Batch 2 (Jobs 3 & 4):** Requires configuration $\{t_3, t_4, t_5\}$.
3. **Batch 3 (Jobs 5 & 6):** Requires configuration $\{t_1, t_5, t_6\}$.

Evaluating the continuous transition cost between these rigid JGP batches:

- **B1 $\rightarrow$ B2:** The magazine transitions from $\{t_1, t_2, t_3\}$ to $\{t_3, t_4, t_5\}$. Tools t_1 and t_2 are dropped to load t_4 and t_5 (2 switches).
- **B2 $\rightarrow$ B3:** The magazine transitions from $\{t_3, t_4, t_5\}$ to $\{t_1, t_5, t_6\}$. Tools t_3 and t_4 are dropped to load t_1 and t_6 (2 switches).

The total cost of the JGP grouping is 4 tool switches. During the transition from B1 to B2, the rigid batch requirement of Batch 2 completely fills the magazine, forcing tool t_1 out, even though it is required for Batch 3.

Conversely, if the U-SSP processes the same sequence of jobs sequentially using the dynamic KTNS policy, the configuration shifts continuously:

- **Job 1 to Job 2:** Magazine remains $\{t_1, t_2, t_3\}$ (0 switches).
- **Job 3:** The magazine drops t_2 (never needed again) and loads t_4. Magazine becomes $\{t_1, t_3, t_4\}$ (1 switch). Tool t_1 is safely retained.
- **Job 4:** Drops t_3, loads t_5. Magazine becomes $\{t_1, t_4, t_5\}$ (1 switch).
- **Job 5:** Drops t_4, loads t_6. Magazine becomes $\{t_1, t_5, t_6\}$ (1 switch).

- **Job 6:** Magazine is already $\{t_1, t_5, t_6\}$. Requirements are met (0 switches).

The total U-SSP cost is **3 tool switches**. By dynamically shifting the configuration and safely retaining tool t_1 in the corner of the magazine, the U-SSP bypasses the rigid capacity choke of the JGP, proving that $Cost_{switches}(\text{U-SSP}) \leq Cost_{switches}(\text{JGP})$.

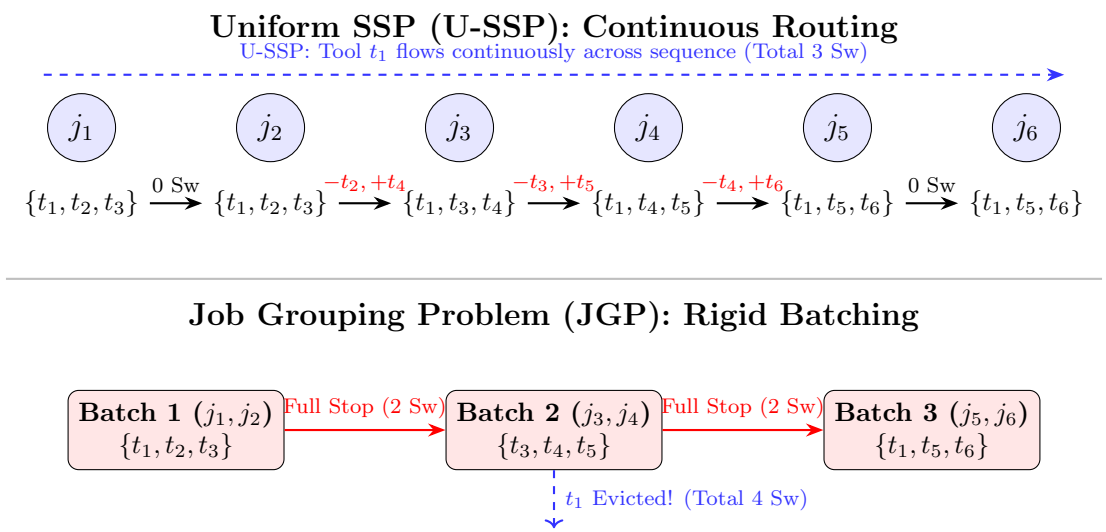

Figure 6: Graph-theoretic contrast between the Job Grouping Problem (JGP) and the Uniform Tool Switching Problem (U-SSP). The JGP acts as a Set Covering problem, mapping jobs to rigid, independent configuration nodes separated by full machine stops where t_1 is unnecessarily evicted. The U-SSP models a continuous routing path, evaluating dynamic edge costs via KTNS and allowing t_1 to flow seamlessly across the sequence.

4.2 Approximation Bounds from JGP to U-SSP

While an optimal JGP solution cannot guarantee an optimal U-SSP sequence, it provides a strict and reliable theoretical upper bound. We can mathematically establish that the optimal JGP solution yields an α-approximation for the U-SSP, where $\alpha = b$ (the magazine capacity).

Theorem 4.1 (α-Approximation)**.** *Let $Cost_{groups}(JGP)$ be the minimum number of groups (machine stops) required to partition the jobs such that no group exceeds the magazine capacity b. Let $Cost_{switches}(U\text{-}SSP)$ be the minimum number of individual tool switches required to process the jobs continuously. Then, the following bound strictly holds:*

$$Cost_{switches}(U\text{-}SSP) \leq b \cdot Cost_{groups}(JGP) \tag{12}$$

Proof. Assume the JGP optimally partitions the job sequence into $k = Cost_{groups}(\text{JGP})$

disjoint groups: $G_1, G_2, \ldots, G_k$. By the definition of the JGP, each group G_i requires a set of tools $T(G_i)$ such that $|T(G_i)| \leq b$.

If we sequence the jobs exactly according to this grouping ($G_1 \to G_2 \to \cdots \to G_k$), the machine must undergo a setup between each group. In the absolute worst-case scenario, the tool requirements for consecutive groups are entirely disjoint ($T(G_i) \cap T(G_{i+1}) = \emptyset$). If this worst-case disjointness occurs, the machine must flush the entire magazine and load exactly b new tools during the transition.

Since there are exactly k groups, there are exactly $k-1$ transitions between them. The initial loading of the magazine at the start of G_1 requires at most b tool insertions. Therefore, the maximum number of individual tool switches across the entire rigid sequence is bounded by:

$$\text{Total Switches} \leq b + b(k-1) = b \cdot k \tag{13}$$

Substituting our initial definitions, we obtain an upper bound for the switches incurred by the rigid JGP grouping:

$$Cost_{switches}(\text{JGP}) \leq b \cdot Cost_{groups}(\text{JGP}) \tag{14}$$

From Section 4.1, we established that evaluating the exact same sequence with the continuous KTNS policy guarantees $Cost_{switches}(\text{U-SSP}) \leq Cost_{switches}(\text{JGP})$ by successfully flipping 0-blocks. Transitivity directly yields our final bound:

$$Cost_{switches}(\text{U-SSP}) \leq Cost_{switches}(\text{JGP}) \leq b \cdot Cost_{groups}(\text{JGP}) \tag{15}$$

Thus, the optimal JGP objective multiplied by the capacity b serves as a strict upper bound (an α-approximation where $\alpha = b$) for the U-SSP objective. □

4.3 Lower Bounds and Polyhedral Cutting Planes

While Theorem 4.1 establishes that the JGP provides a reliable upper bound on the number of tool switches, an exact solution to the U-SSP requires tight lower bounds to effectively prune the search space. Historically, finding strong lower bounds for the U-SSP was a severe computational bottleneck because the linear relaxation of early compact ILP models often yielded a weak lower bound of exactly zero. To overcome this, modern exact approaches rely on state-space bounds and polyhedral cutting planes.

Capacity and Routing Bounds: When constructing a partial sequence of jobs, look-ahead bounds can be dynamically computed to estimate the minimum number of remaining tool switches. Laporte et al. (2004) introduced two highly effective bounding mechanisms

evaluated over the set of unassigned jobs:

1. **The Capacity Bound (LB1):** This bound evaluates the absolute minimum number of tool insertions required to finish the sequence. It is calculated as the total number of unique tools required by all remaining jobs (and not present in the last completed job's toolset), minus the number of currently free slots in the magazine. Mathematically, it is the cardinality of the required remaining tools minus the capacity b Laporte et al. (2004).

2. **The Spanning Tree Bound (LB2):** Because the U-SSP entails a routing component, a valid lower bound on the remaining transition costs can be derived by constructing a minimum spanning tree over the remaining unassigned jobs, utilizing the transition cost metric between their respective required toolsets Laporte et al. (2004).

In a branch-and-bound framework, taking the maximum of these two bounds (or summing them under specific conditions) provides a significantly tighter lower bound than the continuous linear relaxation.

GTSP Cutting Planes: Because we have formally mapped the U-SSP to the GTSP configuration graph (as demonstrated in Section 3), we can directly exploit the tight bounds of the GTSP polytope to strengthen the root node relaxation. The most powerful of these polyhedral cuts are the Generalized Subtour Elimination Constraints (GSECs) Fischetti et al. (1997).

By separating and adding violated GSECs at the root node of a Branch-and-Cut algorithm, the formulation strictly forbids disconnected fractional sub-cycles among the configuration clusters. Fischetti et al. (1997) demonstrated that identifying these violated GSECs via maximum flow separation algorithms and adding them as cutting planes pushes the lower bound extremely close to the optimal integer solution, effectively shattering the "zero-bound" barrier that plagued early compact formulations.

While these exact bounding mechanisms are robust, they rely on having a fully populated state space of valid maximal configurations. Generating these configuration vertices dynamically introduces a profound computational hurdle, inextricably linking the continuous U-SSP back to the subset selection logic of the JGP. We now turn to this bottleneck.

5 Dynamic Node Generation and the Pricing Oracle

As established in Section 3.4, for industrial instances where enumerating the full state space of $\binom{m}{b}$ configurations is intractable, the GTSP configuration graph must be populated dynamically. This section formally dissects the mathematical nature of this dynamic generation

process, strictly differentiating the routing formulation of the U-SSP from traditional column generation schemes, and exposing the severe computational bottleneck that justifies our machine learning interventions.

5.1 Differentiating the Pricing Oracles: SPPRC vs. SUKP

In standard routing problems solved via Branch-and-Price, such as the Vehicle Routing Problem (VRP) or the Prize-Collecting TSP (PCTSP), the Restricted Master Problem (RMP) selects among a pool of *full routes* Barnhart et al. (1998) [1]. Consequently, the pricing subproblem responsible for generating a new column (route) with a negative reduced cost must simultaneously maximize collected dual variables and minimize the continuous routing costs along the edges [2]. This subproblem is universally modeled as the Shortest Path Problem with Resource Constraints (SPPRC) [1].

However, the non-compact formulation of the U-SSP does not generate full routes as columns. Instead, it generates individual *configuration vertices* to incrementally build the GTSP graph. Because adding a new configuration vertex v to the RMP simultaneously requires adding new transition edges $x(u, v)$ connecting it to all existing vertices, this architecture mathematically necessitates Simultaneous Column-and-Row Generation **?** [3].

Within this framework, to generate a promising new configuration vertex that satisfies the covering constraints of the RMP, the pricing oracle evaluates the dual variables π_j associated with each job $j \in J$ [4, 5]. The oracle seeks to find a single valid tool configuration of size b that maximizes the sum of the dual variables for the jobs it can completely process. Because the transition edge costs $d(u, v)$ are only evaluated in the master routing problem once the node is generated, the node-generation oracle itself does not evaluate routing costs.

This subproblem—finding a subset of tools bounded by a capacity constraint to maximize the profit of satisfied jobs—is mathematically identical to the pricing subproblem of the Job Grouping Problem (JGP). As demonstrated by Crama and Oerlemans Crama and Oerlemans (1994) [6, 7], and formally classified in recent literature, this oracle is exactly the Set Union Knapsack Problem (SUKP) Goldschmidt et al. (1994) [7].

5.2 Formalizing the SUKP Pricing Oracle

Let $y_j \in \{0, 1\}$ be a binary variable indicating whether job $j \in J$ is covered by the new configuration [6]. Let $z_t \in \{0, 1\}$ be a binary variable indicating whether tool $t \in T$ is included in the magazine [6]. Given the dual variables $\pi_j \geq 0$ obtained from the RMP's job-covering constraints, the SUKP pricing oracle for the U-SSP is formulated as:

$$\max \quad \sum_{j \in J} \pi_j y_j \tag{16}$$

$$\text{subject to} \quad \sum_{t \in T} z_t = b \tag{17}$$

$$y_j \leq z_t, \quad \forall j \in J, \forall t \in T_j \tag{18}$$

$$y_j \in \{0, 1\}, \quad \forall j \in J \tag{19}$$

$$z_t \in \{0, 1\}, \quad \forall t \in T \tag{20}$$

The objective (16) maximizes the collected dual variables associated with the covered jobs [8]. Constraint (17) strictly enforces the maximal configuration property under the U-SSP assumption by filling the magazine capacity b [8]. The logical linking constraints (18) ensure that a job j can only be claimed as covered ($y_j = 1$) if every single tool t required by that job ($t \in T_j$) is currently loaded in the magazine ($z_t = 1$) [8].

6 The Pricing Oracle and the Set Union Knapsack Problem

The previous sections established a profound link between two classical Combinatorial Optimization problems: the JGP provides the structurally optimal vertices (configurations), while the GTSP routing framework evaluates the continuous transitions between them. However, generating these JGP configurations dynamically within a non-compact Branch-and-Price (B&P) framework introduces a severe computational bottleneck in the form of the Pricing Oracle.

In the non-compact B&P framework, the Restricted Master Problem (RMP) selects configurations, but the **Pricing Subproblem** must iteratively generate new maximal configurations C (where $|C| = b$) with negative reduced costs. Given the dual variables $\pi_i \geq 0$ obtained from the RMP's job-covering constraints, the pricing problem seeks a configuration C that covers a subset of jobs $S \subseteq J$ to maximize the collected duals $\sum_{i \in S} \pi_i$, subject to the magazine capacity $|\bigcup_{i \in S} T_i| \leq b$.

Mathematical Equivalence to the SUKP: This exact mathematical structure is identical to the **Set Union Knapsack Problem (SUKP)** Goldschmidt et al. (1994). In the SUKP, items (jobs) have profits (π_i), and each item requires a set of elements (tools T_i). The weight of a subset of items is defined strictly by the cardinality of the union of their elements.

The SUKP is known to be strongly NP-hard. Exact dynamic programming algorithms for solving the SUKP exhibit an exponential worst-case complexity proportional to:

$$O\left(mnb\sum_{j=1}^{n}2^{|h(j)|}\right) \tag{21}$$

where $h(j)$ denotes the set of elements adjacent to element j in the item-adjacency graph Goldschmidt et al. (1994).

Because the exact formulation of the SSP pricing oracle is fundamentally an SUKP, solving it to optimality at every iteration represents the primary computational barrier preventing the scalability of the exact B&P framework. This exponential complexity barrier explicitly necessitates the development of Machine Learning-based pricing heuristics (such as Neural Column Generation or Deep Reinforcement Learning) to approximate the oracle and accelerate the solver, which we explore in the subsequent sections.

7 Exact Routing for Tractable Instances

As established by the complexity framework, generating the complete configuration graph is computationally viable only for smaller, tightly constrained instances where the combination of m and b does not cause the state space $\mathcal{O}(|J|\binom{m}{b})$ to exceed memory limits. For these tractable instances, we can leverage highly optimized, state-of-the-art routing solvers by applying a secondary polynomial transformation.

Once the overlapping SSP clusters are expanded into mutually exclusive GTSP clusters via the I-N transformation, the problem can be reduced to a standard Asymmetric Traveling Salesman Problem (ATSP). This is achieved through the Noon-Bean transformation (Noon and Bean, 1993), which systematically modifies the intra-cluster and inter-cluster edge weights to logically compel any exact routing algorithm to visit exactly one node per cluster.

By collapsing the generalized problem into an ATSP, the formulation can be passed directly to advanced routing heuristics and exact solvers, such as LKH-3 (Helsgaun, 2015). While this approach yields optimal or near-optimal solutions rapidly for small instances, its reliance on the full, explicit enumeration of the configuration graph renders it intractable for massive industrial applications, necessitating dynamic node generation.

8 Dynamic Node Generation and the True Pricing Bottleneck

For large-scale instances, the exponential size of the vertex set $V(K_p)$ makes it impossible to define the complete problem space upfront. Instead, we transition from the non-compact formulation to a Branch-and-Price paradigm. The problem is decomposed into a Restricted Master Problem (RMP), which operates over a small, generated subset of valid configuration sequences (columns), and a pricing oracle, which dynamically generates new sequences on-the-fly.

The RMP is formulated as a Set Covering problem, selecting a combination of generated sequences to satisfy all job requirements at a minimal transition cost. Let $\pi_j \geq 0$ be the dual variables associated with the covering constraints of the RMP for each job $j \in J$.

The pricing oracle's objective is to find a new sequence of configurations (a column) whose inclusion in the RMP could improve the objective function. Mathematically, it must identify a sequence with a negative reduced cost $\bar{c}$. For a newly generated continuous sequence of configurations C, the reduced cost is calculated as:

$$\bar{c} = \sum_{(u,v)\in E(C)} d(u,v) - \sum_{j\in J(C)} \pi_j \tag{22}$$

where $E(C)$ represents the edges of the generated configuration sequence, $d(u,v)$ is the tool switching transition cost, and $J(C)$ is the subset of jobs covered by the sequence.

Proposition 8.1 (NP-Hardness of the SSP Pricing Oracle)**.** *The pricing problem for the non-compact SSP formulation is strongly NP-hard.*

Proof. The pricing oracle requires identifying a valid sequence of configurations $C \in \mathcal{C}$ that minimizes the reduced cost $\bar{c}$ defined in Equation (22). This requires simultaneously determining an optimal subset of jobs $J(C)$ to cover (to maximize the collected dual variables π_j) while finding the shortest valid tool-switching sequence to process them (to minimize the transition costs $d(u,v)$). This structure perfectly mirrors the Prize-Collecting Traveling Salesman Problem (PCTSP), where dual variables act as collected prizes and switching costs act as routing penalties. Because the PCTSP is inherently strongly NP-hard, identifying the column with the minimum reduced cost for the SSP is also strongly NP-hard. □

It is crucial to distinguish this pricing oracle from the Job Grouping Problem (JGP). The JGP acts as a pure Set Union Knapsack Problem because it assumes the machine is completely halted for simultaneous batch swaps. In contrast, the sequential SSP pricing

oracle must account for continuous transition costs $d(u, v)$ along the edges. Because solving this NP-hard oracle exactly via Mixed-Integer Linear Programming (MILP) at every column generation iteration dominates the total computation time, accelerating the oracle is the primary focus of our future experimental roadmap.

9 Machine Learning Interventions

To overcome the exact pricing bottleneck, we propose integrating Machine Learning (ML) techniques at critical stages of the Branch-and-Price algorithm. By substituting expensive exact computations with rapid, learned approximations, we can accelerate convergence while maintaining global optimality guarantees.

9.1 Pricing Graph Pruning via GNNs

At each iteration, the exact MILP pricing oracle searches for paths across the configuration graph. Because the majority of configurations will never yield a negative reduced cost, searching the entire space is highly inefficient. We propose training a Graph Neural Network (GNN) to predict the likelihood of a configuration belonging to an optimal column based on the current dual variables π_j. By filtering out low-probability nodes, the GNN drastically prunes the pricing graph, reducing the search space for the exact MILP solver in subsequent iterations.

9.2 Reinforcement Learning for Column Selection

A classic approach in column generation is to add multiple columns at each iteration rather than just the single most negative one. However, highly correlated columns lead to degeneracy in the RMP. The RMP naturally forms a bipartite graph between jobs and configurations. We can train a Deep Reinforcement Learning (RL) agent to evaluate this bipartite state and select a diverse batch of mutually disjoint configurations. Injecting varied, high-quality columns into the RMP at each iteration mitigates degeneracy and minimizes the total number of column generation iterations required. Furthermore, adapting the Ryan-Foster branching rule—originally designed for Set Partitioning (equality constraints)—to handle the SSP's Set Covering (inequality constraints) structure provides a fertile ground for RL branching agents to discover optimal branching variables.

9.3 Feasibility Checking via Sequence Models

A major computational burden within the pricing oracle is verifying whether a generated sequence of jobs respects the tool magazine capacity across all transitions. **?** implement a Neural Column Generation (NCG) pipeline where an Attention-based sequence model predicts the feasibility of columns prior to invoking an expensive exact feasibility checker. By rejecting structurally infeasible sequences early using the sequence model, we bypass unnecessary computational overhead.

9.4 Regression for Bound Prediction

Finally, the exact MILP solver used in the pricing oracle can be significantly accelerated if provided with a tight upper bound for its objective function. Following the framework proposed by Václavík et al. (2018), we will deploy a Regression model to predict the expected minimum reduced cost at each iteration based on historical dual variable values. Passing this predicted bound to the MILP solver enables aggressive pruning of its own search tree, drastically reducing the time spent proving optimality.

10 Experimental Roadmap

The theoretical properties established in this report pave the way for a highly structured experimental plan. We outline a phased integration of Machine Learning methods to progressively accelerate the exact algorithm, summarized in Table 1.

Phase	Description	RMP Init.	Pricing Graph	Oracle Generator	Column Select.	Exact Fallback
0	Baseline	Random	Full Graph	Exact MILP	Greedy (Std)	Exact MILP
1	OR Enhancements	Multi-Start Heuristic	Full Graph	Exact MILP	Greedy (Std)	Exact MILP
2	Exact Acceleration	Best from Phase 1	GNN-Pruned	Exact MILP	Greedy	Regress. Bound
3	Neural Generators	Best from Phase 1	GNN-Pruned	MLPH / GNN	Diverse-M (RL)	Regress. Bound
4	SOTA Synthesis	Best from Phase 1	GNN-Pruned	Attn-Sequence	Deep RL	Regress. Bound

Table 1: Experimental Roadmap for ML-Accelerated Exact Branch-and-Price.

Phase 0 establishes the computational baseline using a standard Branch-and-Price framework. Phase 1 introduces pure Operations Research (OR) enhancements, utilizing advanced multi-start heuristics to seed the RMP. Phase 2 introduces exact ML accelerators, deploying GNNs to prune the pricing graph and Regression models to predict bounds for the MILP solver. Phase 3 replaces the exact oracle entirely with Neural Generators (MLPH/GNN)

for the bulk of the iterations. Finally, Phase 4 represents the state-of-the-art synthesis: using Sequence Models to rapidly generate columns, Deep RL to select a disjoint batch of them, and Regression Bound Prediction to accelerate the final, mandatory exact proof of optimality.

11 Conclusion

This report formalized the mathematical properties of a non-compact, configuration-based formulation for the Uniform Tool Switching Problem (SSP). By establishing the structural subsumption of the Generalized Traveling Salesman Problem (GTSP) within the SSP under the strict Disjointness Condition, we successfully proved that the metric barrier prevents a generalized polynomial reduction. This complexity framework mathematically justifies a bifurcated algorithmic approach: while tractable instances can be reduced to the Asymmetric TSP and solved via highly optimized routing heuristics, industrial-scale instances necessitate dynamic node generation. Building upon this rigorous foundation, we proposed a progressive Experimental Roadmap designed to leverage Machine Learning—specifically Graph Neural Networks, Deep Reinforcement Learning, and Regression—to bypass the NP-hard pricing bottleneck within an exact Branch-and-Price paradigm.

References

Cynthia Barnhart, Ellis L. Johnson, George L. Nemhauser, Martin W. P. Savelsbergh, and Pamela H. Vance. Branch-and-price: Column generation for solving huge integer programs. *Operations Research*, 46(3):316–329, 1998.

Daniele Catanzaro, Luis Gouveia, and Martine Labbé. Improved integer linear programming formulations for the job sequencing and tool switching problem. *European Journal of Operational Research*, 244(3):766–777, 2015.

Rafael Colares and Annegret Wagler. Exact formulations for the job sequencing and tool switching problem. *Working Paper*, 2026.

Yves Crama and A. G. Oerlemans. A column generation approach to job grouping for flexible manufacturing systems. *European Journal of Operational Research*, 78(1):58–80, 1994.

Matteo Fischetti, Juan José Salazar-González, and Paolo Toth. A branch-and-cut algorithm for the symmetric generalized traveling salesman problem. *Operations Research*, 45(3): 378–394, 1997.

O. Goldschmidt, D. Nehme, and G. Yu. Note on the set-union knapsack problem. *Naval Research Logistics (NRL)*, 41(6):833–842, 1994.

Keld Helsgaun. Solving the equality generalized traveling salesman problem using the lin–kernighan–helsgaun algorithm. *Mathematical Programming Computation*, 7(3):269–287, 2015.

Gilbert Laporte, Juan José Salazar-González, and Frédéric Semet. Exact algorithms for the job sequencing and tool switching problem. *IIE Transactions*, 36(1):37–45, 2004.

Yao-Nan Lien, Eva Ma, and Benjamin W. S. Wah. Transformation of the generalized traveling-salesman problem into the standard traveling-salesman problem. *Information Sciences*, 74(1-2):177–189, 1993.

Charles E. Noon and James C. Bean. An efficient transformation of the generalized traveling salesman problem. *INFOR: Information Systems and Operational Research*, 31(1):39–44, 1993.

Caroline Privault and Gerd Finke. Modelling a tool switching problem on a single nc-machine. *Journal of Intelligent Manufacturing*, 6(2):87–94, 1995.

Christopher S. Tang and Eric V. Denardo. Models arising from a flexible manufacturing machine, part i: Minimization of the number of tool switches. *Operations Research*, 36(5): 767–777, 1988.

Robin Václavík, Antonín Novák, Přemysl Šůcha, and Zdeněk Hanzálek. Accelerating the branch-and-price algorithm using machine learning. *European Journal of Operational Research*, 271(3):1055–1069, 2018.